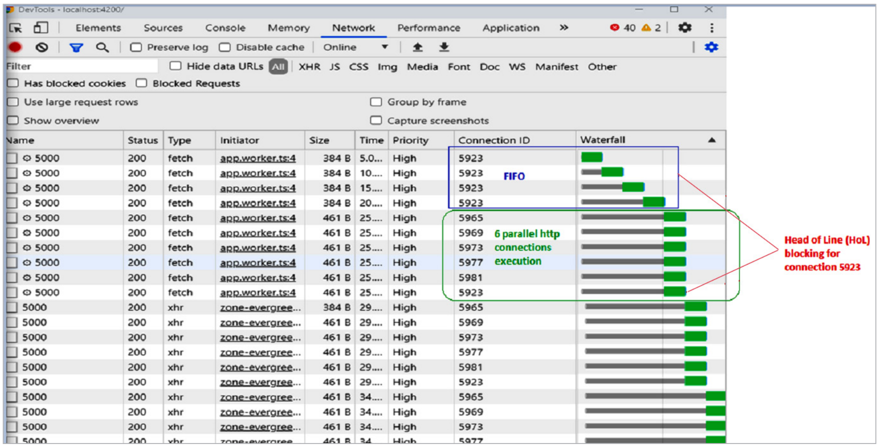


Figure 5: Problems with http1.1

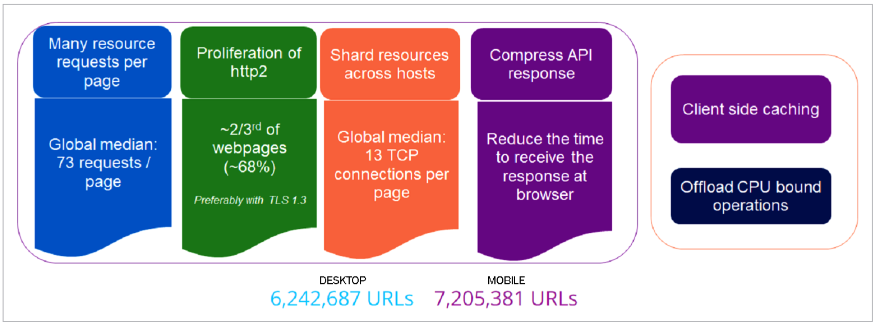
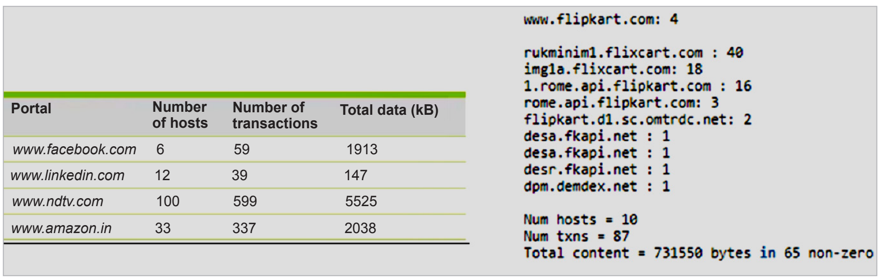


Figure 6: Web scalability tenets [*Reference: https://httparchive.org/reports/state-of-the-web?start=earliest&end=latest&view=list*]



Portal	Number of hosts	Number of transactions	Total data (kB)
www.facebook.com	6	59	1913
www.linkedin.com	12	39	147
www.ndtv.com	100	599	5525
www.amazon.in	33	337	2038

```
www.flipkart.com: 4

rukminim1.flixcart.com : 40
img1a.flixcart.com: 18
1.rome.api.flipkart.com : 16
rome.api.flipkart.com: 3
flipkart.d1.sc.omtrdc.net: 2
desa.fkapi.net : 1
desa.fkapi.net : 1
desr.fkapi.net : 1
dpm.demdex.net : 1

Num hosts = 10
Num txns = 87
Total content = 731550 bytes in 65 non-zero
```

Figure 7: Common websites [Reference: *https://opensourceforu.com/2020/03/the-evolution-of-web-protocols-2/*]

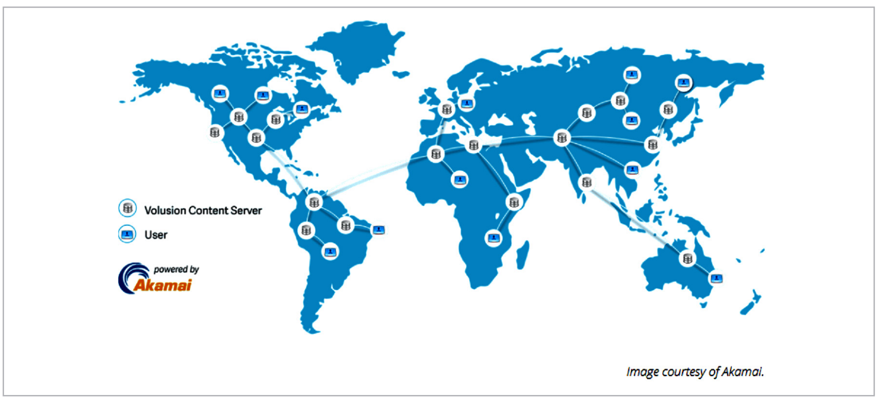


Figure 8: Akamai CDN deployment

If you want to process multiple requests for a higher throughput requirement of the application, the protocol restricts you to only six concurrent requests at most, at a given time. Note that the http-1.1 protocol RFC does not say a lot about the number of concurrent requests. Most browsers support up to six requests by default. Some browsers can support up to eight. The problem with http1.1 is shown in Figure 5 as the waterfall timing diagram of an application, captured in the Chrome dev tool (press F-12 in the Chrome browser) network tab.

Web scalability tenets

Let us look at the various tenets of Web scalability, in a nutshell (Figure 6).

According to publicly available data from the 'state-of-the-web' repository, that scanned over 6 million desktop Web apps and 7 million mobile apps, the following are the salient observations:

1. The average number of requests per page is 73 – quite a large number.
2. Only one-third of the Web pages are still in legacy http-1.1. Others have moved to http2.
3. Average connections per page are 13, which are more than six (the browser limit for http1.1).
4. Compression of the response is commonplace at the server side; the browser handles the decompression as per the HTTP protocol support.
5. In the client context, client-side caching (layer-7 as well as layer-4), along with usage of Web workers to offload the CPU intensive browser operations into another worker thread(s), is often deployed. The worker thread concept is particularly interesting as the underlying JavaScript execution is single-threaded in a browser.